

Trabalho Prático 1

Scripting Processamento de Linguagem Natural

Tema: Música

Jéssica Cunha, pg60267

Agenda

- 01 Objetivo & Tema
- 02 Fontes de Dados (3 fontes)
- 03 Arquitetura & Pipeline (run_all.py)
- 04 Recolha: MuseScore (web scraping)
- 05 Recolha: Wikipédia (requests + BS4)
- 06 Recolha: PDF (pdftotext)
- 07 Modelos de N-gramas (código)
- 08 NER com spaCy (código)
- 09 Geração LaTeX & Anotação (código)
- 10 Outputs: 3 artigos PDF
- 11 Conclusão

Objetivo & Tema



Objetivo do TP1

- 1. Recolher textos**
de 3+ fontes sobre um tema
- 2. Construir modelos de linguagem**
baseados em n-gramas
- 3. Aplicar NER**
com spaCy (pt_core_news_sm)
- 4. Gerar artigos LaTeX**
anotados com entidades
- 5. Compilar PDFs**
automaticamente via pdflatex

Tema: Música

Domínio com vocabulário técnico rico:

- Teoria musical (escalas, acordes, claves)
- Instrumentos e notação
- Personalidades históricas
- Termos especializados em PT e EN

Ideal para testar n-gramas (bigramas musicais frequentes) e NER (compositores, locais, organizações).

Fontes de Dados

3fontesheterogêneas—web,API,PDF—todassobre música



MuseScore Glossário

URL: handbook.musescore.org

Método: requests + BS4

Estrutura JSON:

```
Dict[letra → Dict[termo → {def,  
var, flag}]]
```

HTML → JSON



Wikipédia Música

URL: pt.wikipedia.org/wiki/Música

Método: requests + BS4

Estrutura JSON:

```
Dict[secção → List[parágrafo]]
```

HTML → JSON



Manual Teoria Musical(LXPRO)

URL: <https://www.lxpro.pt/img/pdf/manual%20teoria%20musical.pdf>

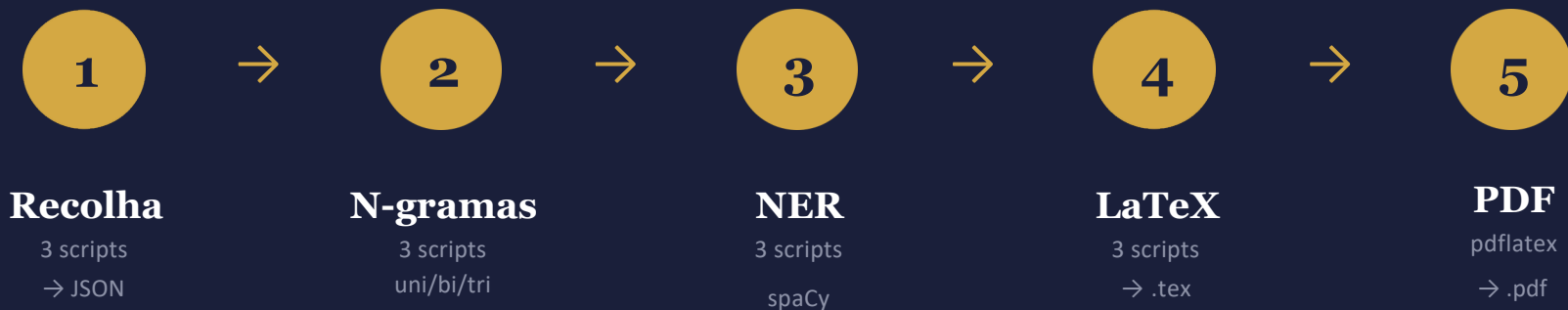
Método: pdftotext + regex

Estrutura JSON:

```
Dict[título → List[parágrafo]]
```

PDF → TXT → JSON

Arquitetura & Pipeline



>_ run_all.py

Script automatizado que executa sequencialmente cada fase para as 3 fontes (musescore/, wikipedia/, pdf/), com feedback visual e tratamento de erros. Um único comando corre todo o pipeline.

Recolha: MuseScore (Web Scraping)

recolha_musescore.py —requests + BeautifulSoup → musescore_glossario.json

MuseScore Glossário

recolha_musescore.py — requests + BeautifulSoup → Extrai termos (A–Z) com definições, variante AE/BE e flag MuseScore-específico → musescore_glossario.json

Output JSON

Exemplo:

```
{ "A":  
  {  
    "Acicatura":  
      {  
        "definicao": "...",  
        "variante": null,  
        "musescore_especifico": false  
      }, ...  
    },  
  "B": {...}, ... }
```

Funcionalidades:

- Deteção AE/BE (American/British)
- Flag MuseScore-específico(*)
- Limpeza de texto (regex)
- Filtragem de letras

Recolha:Wikipédia

recolha.py — requests + BeautifulSoup → wikipedia_musica.json

Wikipédia -Música

recolha.py — requests + BeautifulSoup → Extrai secções (h2/h3) e parágrafos (<p>), filtra secções inúteis(Referências, Ver também, etc.) → wikipedia_musica.json

Detalhes

- User-Agent para evitar bloqueio
- Limpa refs numéricas[1]
- Limpa refs letra[a]
- Limpa botões[editar]
- Filtra secções inúteis
- Filtra parágrafos < 30 chars
- Output: Dict[secção→parágrafos]

Recolha: Manual PDF (pdftotext)

recolha_manual.py — pdftotext → limpeza → manual_teoriamusical.json

Manual de Teoria Musical (PDF)

recolha_manual.py — lê .txt gerado por pdftotext →

limpa cabeçalhos/números de página → agrupa em parágrafos (>30 chars, com '.')

→ manual_teoriamusical.json

Limpeza aplicada

1. Remove cabeçalhos repetidos("LXPRO", título)
2. Remove números de página isolados
3. Remove marcadores ~N~
4. Agrupa linhas consecutivas em parágrafos
5. Filtra parágrafos <30 chars
6. Exige ponto final (descarta índice)

Modelos de N-gramas —Código

ngrams_muscore.py / ngrams.py / ngrams_manual.py —mesma lógica em cada fonte

```
def tokenizar(texto):
    texto= texto.lower()
    texto= re.sub(r"^[^wÀ-ú\s]", " ", texto) # mantém  acentos
    return [t for t in texto.split() if len(t) >= 2]

def build_ngrams(tokens, n):
    return Counter(tuple(tokens[i:i+n]) for i in range(len(tokens)-n + 1))

# Score de frase= log-prob médiados bigramas
def score_frase(tokens):
    if len(tokens) < 8: return float('-inf')
    score = sum(log(p2(tokens[i], tokens[i+1]))) for i
    in range(len(tokens) - 1)
    return score / (len(tokens) - 1)

# Top 3 frases mais representativas
frases_scored = sorted([
    (score_frase(t), f) for f, t in in frases_tokenizadas], reverse=True)
top3 = [f for _, f in frases_scored[:3]]
```

Conceitos-chave

Tokenização

Lowercase + remove pontuação + filtro ≥ 2 chars.
Preserva acentos PT.

Laplace smoothing

Evita probabilidades zero para bigramas nunca vistos. $P(w_2 | w_1) = (C_1 + 1) / (C_1 + V)$

Score de frase

Média da log-prob dos bigramas. Filtro mínimo: 8 tokens (evita frases curtas).

Output

Top3 frases vão para o abstract LaTeX de cada artigo.

NER com spaCy —Código

ner_muscore.py / ner.py / ner_manual.py —extração e filtragem de entidades

```
nlp = spacy.load("pt_core_news_sm")
doc = nlp(texto_completo)

def is_valida(ent):
    texto= ent.text.strip()
    if len(texto) <= 2: return False # genéricas completas
    if texto.isupper(): return False # siglas
    if len(texto.split()) > 5: return False # siglas
    ruído= {'veja', 'consulte', 'inserindo', 'indica',
            'usado', 'trabalhando', 'criar', 'preencher'}
    return texto.lower().split()[0] not in ruído
```

Filtragem de ruído

Filtragem aplicada:

- Entidades ≤ 2 caracteres
- Entidades > 5 palavras
- Siglas all-caps genéricas
- Verbos comuns ("veja", "consulte")

O modelo pt_core_news_sm gera muitos falsos positivos em texto musical ("Clave" → LOC, "Música" → LOC).

LaTeX & PDFs



artigo_musescore.pdf

Glossário A–Z com entidades anotadas inline



artigo_wikipedia_musica.pdf

Artigo organizado por secções da Wikipédia



artigo_manual.pdf

Texto do manual com anotação automática

Cada artigo contém: título + abstract (top 3 frases) + fonte + corpo anotado com `\textbf{entidade}\textsc{[TIPO]}` + bibliografia

Outputs: 3 Artigos PDF

Gerados automaticamente via pdflatex

artigo_musescore.pdf

Top frase: *"Sinal(#) que indica que a altura de uma nota deve ser aumentada em um semitom, veja acidentais e armadura de clave."*

Entidades: MuseScore, Werner Schweer, E-mu Systems, MIDI

artigo_wikipedia_musica.pdf

Top frase: *"A história da música é o estudo das origens e evolução da música ao longo do tempo."*

Entidades: Mozart, Brasil, Grécia, Jean-Jacques Nattiez

artigo_manual.pdf

Top frase: *"Fá maior tem um valor de -1 (um bemol)."*

Entidades: Música, Pauta, Clave, Pentagrama, Dórico

Conclusão

Pipeline completo de PLN: recolha automatizada → modelos de linguagem → anotação semântica → produção de artigos.

- ✓ Modelos de bigramas com Laplace smoothing selecionam frases representativas
- ✓ NER com spaCy + filtragem de ruído enriquece textos com anotações semânticas
- ✓ 3 artigos LaTeX com abstract, corpo anotado e bibliografia gerados automaticamente